

Section 86 — Cut Trees And Matroid Intersection

Compress repeated optimization into a tree or an exchange graph

This lesson develops A–C. Problem D is editorial-only.

Problem A: Gomory–Hu trees

For an undirected graph, one weighted tree represents every pair minimum cut: the answer for u, v is the minimum edge on their tree path. Build it using $n-1$ maximum flows. Each residual source side reparents later vertices sharing the same current parent; the parent-swap correction preserves cuts nested through that parent.

Problem B: exchange-graph augmentation

Maintain a set independent in two matroids. Outside elements addable to matroid 1 are BFS sources; elements addable to matroid 2 are sinks. Arcs alternate between exchanges preserving matroid 1 and exchanges preserving matroid 2. Flipping membership along a source-to-sink path grows the common set by one. The matroid-intersection theorem says no augmenting path is an optimality certificate.

Problem C: graphic independence oracles

A graphic matroid declares an edge set independent exactly when it is a forest, testable by DSU. Two endpoint maps therefore give two graphic oracles on the same selectable elements. This remains polynomial for two matroids; an arbitrary third constraint would not be a harmless extension.

Reading one Gomory–Hu step

Suppose vertex s currently has parent t . Compute a minimum s – t cut and inspect the vertices reachable from s in the final residual graph. Every later vertex v on that side whose current parent is t is reparented to s . The new tree edge (s, t) stores the cut value.

The subtle parent swap occurs when the old parent of t lies on the source side. Then the new cut must be inserted above, rather than below, t ; copying the appropriate old weight preserves previously represented cuts. Draw the three involved vertices before coding this case. The algorithm is not merely collecting $n-1$ unrelated cuts: the reparenting is what makes their source sides form a tree representation.

Deriving the matroid exchange graph

Let I already be independent in both matroids. To grow it, we may have to add an outside element and remove an inside element several times. Encode exactly those legal one-element repairs:

- source to outside x if $I+x$ is independent in the first matroid;
- outside x to inside y if $I-y+x$ is independent in the first;
- inside y to outside x if $I-y+x$ is independent in the second;
- outside x to sink if $I+x$ is independent in the second.

An alternating source-to-sink path tells us which memberships to toggle. Its first and last additions survive, while intermediate additions and removals balance, so the cardinality increases by one. More importantly, the arc directions make the two independence arguments alternate; ignoring their direction is a common but fatal simplification.

Oracle design and validation

For a graphic matroid, rebuild a DSU from the candidate set and reject the first edge whose endpoints are already connected. This simple oracle may be fast enough under the given limits and is much easier to verify than a dynamic forest. For a partition matroid, maintain or recompute group counts.

Before optimizing an oracle, compare the whole algorithm with exhaustive subset enumeration on tiny ground sets. Include parallel edges, loops, already-optimal sets, an augmenting path requiring several exchanges, and instances that are independent in either matroid alone but not both.

Exercises

A–C develop cut-tree compression and matroid exchange. D requires directed cycle contraction.